Блокировка чтения-записи

Нужна своя блокировка, которая пускает к данным сразу нескольких читателей, а писателя — только одного и только когда читателей нет. Проверка — отсортированный односвязный список из `pth_ll_rwl.c`. Та же программа собирается второй раз с библиотечной `pthread_rwlock_t`, и времена сравниваются.

Запуск:

```
./ll_my  <потоки> [ключей_в_main операций доля_поиска доля_вставки]
./ll_lib <потоки> [ключей_в_main операций доля_поиска доля_вставки]
```

Без четырёх последних чисел программа спрашивает их с клавиатуры, как исходный файл. Доля удалений равна `1 - поиск - вставка`.

Как устроена блокировка

Структура — из пункта 2.3.2 методички: мьютекс, две условные переменные, счётчик текущих читателей, счётчики ожидающих чтение и запись, флаг писателя.

Мьютекс защищает только эти поля. Обход списка идёт уже после того, как `rdlock` или `wrlock` отпустил мьютекс. Иначе читатели стояли бы в очереди на мьютексе и параллельного чтения не было бы.

`pthread_cond_wait` стоит в цикле `while` и проверяет само условие, а не код возврата. Условная переменная может разбудить поток без причины. `pthread_cond_wait` при этом возвращает 0, как и при нормальном пробуждении, и на время сна сам отпускает мьютекс.

Правило такое. Новый читатель ждёт, если писатель уже внутри или хотя бы один писатель стоит в очереди. Иначе сплошной поток поисков не подпустит вставку. Писатель ждёт, пока `readers == 0` и флаг писателя сброшен.

`unlock` будит кого-то только когда внутри уже никого нет. Писатель один, ему хватает `signal`. Читателей несколько, их поднимает `broadcast`. Если в очереди есть писатель, будится он, а не читатели.

Счётчики операций (`member_count` и остальные) по-прежнему складываются под отдельным `count_mutex` в конце работы потока. К списку этот мьютекс не относится.

Список

`Insert`, `Member` и `Delete` не изменились. Поиск берёт `rdlock`, вставка и удаление — `wrlock`. Случайные ключи даёт `my_rand`: у каждого потока свой `seed`, общего состояния нет. Начальные ключи вставляет главный поток до `pthread_create`. После `join` программа проверяет, что список строго возрастает и без повторов.

Где и как мерилось время

Процессор	Intel Core i5-13420H, в WSL видно 6 ядер и 12 потоков
Память	3.7 ГБ, выделено WSL2
Система	Ubuntu 22.04, ядро 6.6.87.1-microsoft-standard-WSL2
Сборка	gcc 11.4.0, флаги `-O2 -pthread`

Время берёт макрос `GET_TIME` из `timer.h`: `gettimeofday`, секунды с долями. Замер стоит вокруг `pthread_create` и `pthread_join`. Вставка начальных ключей в замер не входит.

Каждая пара «реализация, число потоков» запускалась 5 раз. В таблицы положен минимум. Рядом в `results/bench.csv` есть и среднее. Ускорение `S = T(1) / T(p)` считается отдельно для каждой реализации: у них разное время на одном потоке.

Число операций делится на число потоков нацело (12000 и 24000 делятся на 1, 2, 4, 6, 8 и 12), поэтому суммарный объём работы при разном p один и тот же.

Две нагрузки:

- только поиск: 8000 ключей, 12000 операций, доля поиска 1;
- смесь: 1000 ключей, 24000 операций, поиск 0.90, вставка 0.01, удаление 0.09.

На одинаковом числе потоков и одинаковых долях обе программы делают одинаковое число поисков, вставок и удалений. Список после прогона остаётся отсортированным.

Только поиск

8000 ключей, 12000 поисков:

Потоки	Своя, с	S	pthread_rwlock, с	S
1	0.6791	1.00	0.7265	1.00
2	0.3123	2.17	0.3395	2.14
4	0.1788	3.80	0.1693	4.29
6	0.1497	4.54	0.1306	5.56
8	0.1108	6.13	0.1136	6.39
12	0.1150	5.90	0.1277	5.69

У библиотечной блокировки минимум на 4 потоках даёт S = 4.29. Среднее тех же пяти запусков — 0.196 с против 0.766 с на одном потоке, S = 3.91. Запас над четырьмя — это разброс минимума. Список из 8000 узлов занимает около 128 КБ и помещается в L2, обход хорошо делится между ядрами.

Обе реализации здесь близки. На одном потоке своя чуть быстрее: `rdlock` — это захват мьютекса, увеличение счётчика и отпуск. На 12 потоках ускорение около 6. Дальше 6 потоков прирост меньше: у машины в WSL 6 ядер, остальные потоки — гиперпоточность.

Писателей в этом прогоне нет, поэтому цикл ожидания ни разу не засыпает. Долгая часть операции — обход списка — идёт без мьютекса, сразу в нескольких потоках.

Смесь с записью

1000 ключей, 24000 операций, 90% поиск, 1% вставка, 9% удаление:

Потоки	Своя, с	S	pthread_rwlock, с	S
1	0.0547	1.00	0.0500	1.00
2	0.1234	0.44	0.2023	0.25
4	0.3242	0.17	0.2282	0.22
6	0.5182	0.11	0.2322	0.22
8	0.5791	0.09	0.2268	0.22
12	0.6693	0.08	0.2460	0.20

Здесь дополнительные потоки время не уменьшают. Удаление и вставка берут исключительную блокировку, а пока писатель ждёт, новые читатели тоже стоят. Список короткий,

обход дешевле, чем очередь на мьютексе блокировки. Чем больше потоков, тем чаще они будят друг друга через `signal` и `broadcast`.

Библиотечная блокировка после двух потоков выходит на полку около 0.23 с. Своя растёт и дальше, до 0.67 с на 12 потоках. `pthread_rwlock` считает читателей атомарно и засыпает через futex. Своя на каждое взятие и отпуск захватывает один мьютекс.

На одном потоке обе укладываются в 0.05 с: ждать некого, разница — это стоимость одного захвата.

Вывод

На чистом поиске своя блокировка делает то, что от неё требуется: несколько потоков читают список одновременно, ускорение до 6 ядер почти такое же, как у `pthread_rwlock`. На смеси с удалением выигрыша нет ни у одной реализации, а своя проседает сильнее: приоритет писателя и один общий мьютекс на счётчики держат все потоки на короткой операции.

Код блокировки — `my_rwlock.c`. Список и замер — `pth_ll_rwl.c`, флаг сборки `-DUSE_MY_RWLOCK` подставляет свою реализацию вместо библиотечной.